The following listing contains the test code for the musician view, including the database setup. Add the code to bands/tests.py to try the test out.

Listing 8.2 A test case for the musician view

```
# bands/tests.py
from datetime import date
from django.test import TestCase
from bands.models import Musician

class TestBands(TestCase):
    def setUp(self):
        self.musician = Musician.objects.create(first_name="First",
            last_name="Last", birth=date(1900, 1, 1))

    def test_musician_view(self):
        url = f"/bands/musician/{self.musician.id}/"
        response = self.client.get(url)

        self.assertEqual(200, response.status_code)
        self.assertEqual(response.context['musician'].id,
            self.musician.id)
        self.assertIn(self.musician.first_name,
            str(response.content))
```

The .setUp() method is called at class creation and is typically used to create database entries.

Fetch the response for the musician view.

Validate that the musician in the view's context is the one in our test.

Validate that the musician's name shows up somewhere in the rendered HTML.

With your new test in place, use the `test` management command to execute it. I have a funny story for you. Ok, not funny "haha," more like funny "what's that smell." Writing this example test actually helped me find a bug. The exercise in chapter 7 adds features to the musician view that require use of the `UserProfile` object. This view doesn't require a user to be logged in, though, and the anonymous user doesn't have an associated `UserProfile` object. Writing this test caused my initial version of the view to crash, and I had to go back and put a check for the profile's existence in the code. See? Tests are important.

In addition to the `.setUp()` method, there is also a `.setUpClass()` class method. This gets called once when the `TestClass` is created. It may be more efficient to do your initial configuration there. If you're using the `.setUpClass()` method, it is important to remember two things: it must be wrapped with the `@classmethod` decorator, as it is a class method, and you must call the parent's method using `super().setUpClass()`, or you will get an error.

Going along with `.setUp()` and `.setUpClass()` are `.tearDown()` and `.tearDownClass()`. The first gets called each time a test method completes, and the second gets called when the class finishes. The same rules about the decorator and call to `super()` apply to `.tearDownClass()`. The tear-down methods are useful in Python to do things like database clean-up. As Django already does this for you, you won't find you need them as often when writing tests for Django code. The complete flow of creating a test class and running the setup and tear-down for each test is shown in figure 8.4.